

write_env and unstoppable Sender Adaptors

Document #:	P3284R3
Date:	2025-02-13
Project:	Programming Language C++
Audience:	Library Working Group
Reply-to:	Eric Niebler < eric.niebler@gmail.com >

Contents

- [1 Introduction](#)
- [2 Executive Summary](#)
- [3 Description](#)
 - [3.1 write_env](#)
 - [3.1.1 Example: write_env](#)
 - [3.2 unstoppable](#)
 - [3.2.1 Example: unstoppable](#)
- [4 Proposed Wording](#)
- [5 References](#)

§ 1 Introduction

This paper proposes to add two new sender adaptor algorithms to the `std::execution` namespace, targeting C++26: `write_env` and `unstoppable`. These adaptors were originally proposed as part of [P3175R3] but were split out into their own paper so that the higher priority items in P3175 could advance more quickly.

§ 2 Executive Summary

Below are the specific changes this paper proposes:

1. Add a new uncustomizable adaptor `write_env` for writing values into the receiver's execution environment. Use `write_env` in the implementation of the `on` algorithm and to simplify the specification of the `let_` algorithms.

2. Add an uncustomizable unstoppable adaptor that is a trivial application of `write_env`: it sets the current stop token in the receiver's environment to a `never_stop_token`.

§ 3 Description

[P3175R3] proposed some changes to the `std::execution::on` algorithm, the specification of which was made simpler by the addition of some additional adaptors. Those adaptors were general and useful in their own right, so P3175R3 suggested they be added to `std::execution` proper. The conservative approach was to make them exposition-only, and that is how things currently stand in the working draft.

The author still feels like those adaptors are worthy of standardization. This paper proposes adding them.

The adaptors in question are as follows:

§ 3.1 `write_env`

A receiver has an associated “execution environment”, which is an unstructured, queryable key/value store. It is used to pass implicit parameters from parent operations to their children. It is occasionally useful for a sender adaptor to explicitly mutate the key/value store so that child operations see different values for environment queries. The `write_env` sender adaptor is used for that purpose.

`write_env` is a customization point object, although it is not actually customizable. It accepts a sender `sndr` and an execution environment `env`, and it returns a new sender that stores `sndr` and `env`. When that sender is connected to a receiver `rcvr`, it returns the result of connecting `sndr` with a receiver that adapts `rcvr`. The environment of that adapted receiver is the result of joining `env` with `rcvr`'s environment. The two environments are joined such that, when the joined environment is queried, `env` is queried first, and if `env` doesn't have a value for that query, the result of `get_env(rcvr)` is queried.

§ 3.1.1 Example: `write_env`

One example of where `write_env` might be useful is to specify an allocator to be used by child operations. The code might look like this:

```
// Turn a query object and a value into a queryable environment
// (see [P3325R2]):
template <class Query, class Value>
struct prop {
    Query query;
    Value value;
    decltype(auto) query(Query) const noexcept { return (value); }
};

// Adapts a sender so that it can use the given allocator:
```

```

struct with_allocator_t {
    template <std::execution::sender Sndr, class Alloc>
    auto operator()(Sndr sndr, Alloc alloc) const {
        return std::execution::write_env(sndr, prop(std::get_allocator,
            alloc));
    }

    template <class Alloc>
    auto operator()(Alloc alloc) const {
        return std::execution::write_env(prop(std::get_allocator, alloc));
    }
};

constexpr with_allocator_t with_allocator{};

```

The `with_allocator` adaptor might be used to parameterize senders produced by a third-party library as follows:

```

namespace ex = std::execution;

// This returns a sender that does some piece of asynchronous work
// created by a third-party library, but parameterized with a custom
// allocator.
ex::sender auto make_async_work_with_alloc() {
    ex::sender auto work = third_party::make_async_work();

    return with_allocator(std::move(work), custom_allocator());
}

```

The sender returned by `third_party::make_async_work` might query for the allocator and use it to do allocations:

```

namespace third_party {
    namespace ex = std::execution;

    // A function that returns a sender that generates data on a special
    // execution context, populate a std::vector with it, and then completes
    // by sending the vector.
    constexpr auto _populate_data_vector =
        []<class Allocator>(Allocator alloc) {
            // Create an empty vector of ints that uses a specified allocator.
            using IntAlloc = std::allocator_traits<Allocator>::template
                rebind_alloc<int>;
            auto data = std::vector<int, IntAlloc>{IntAlloc{std::move(alloc)}};

            // Create some work that generates data and fills in the vector.
            auto work = ex::just(std::move(data))
                | ex::then([](auto data) {
                    // Generate the data and fill in the vector:
                    data.append_range(third_party::make_data())
                    return data;
                });

            // Execute the work on a special third_party execution context:
            // (This uses the `on` as specified in P3175.)

```

```

    return ex::on(third_party_scheduler(), std::move(work));
};

// A function that returns the sender produced by
// `_populate_data_vector`,
// parameterized by an allocator read out of the receiver's environment.
ex::sender auto make_async_work() {
    return ex::let_value(
        // This reads the allocator out of the receiver's execution
        // environment.
        ex::read_env(std::get_allocator),
        _populate_data_vector
    );
}
}

```

§ 3.2 unstoppable

The unstoppable sender adaptor is a trivial application of `write_env` that modifies a sender so that it no longer responds to external stop requests. That can be of critical importance when the successful completion of a sender is necessary to ensure program correctness, *e.g.*, to restore an invariant.

The unstoppable adaptor might be implemented as follows:

```

inline constexpr struct unstoppable-t {
    template <sender Sndr>
    auto operator()(Sndr sndr) const {
        return write_env(std::move(sndr), prop(std::get_stop_token,
            never_stop_token()));
    }

    auto operator()() const {
        return write_env(prop(std::get_stop_token, never_stop_token()));
    }
} unstoppable {};

```

§ 3.2.1 Example: unstoppable

In the following example, some asynchronous work must temporarily break a program invariant. It uses `unstoppable` and a hypothetical `finally` algorithm to restore the invariant. `finally` runs a predecessor sender, saves its results, runs another sender, and then propagates saved results of the predecessor.

```

namespace ex = std::execution;

ex::sender auto break_invariants(auto&... values);
ex::sender auto restore_invariants(auto&... values);

// This function returns a sender adaptor closure object. When applied
// to
// a sender, it returns a new sender that breaks program invariants,
// munges the data, and restores the invariants.

```

```

auto safely_munge_data( ) {
    return ex::let_value( [] (auto&... values) {
        return break_invariants(values...)
        | ex::then(do_munge) // the invariants will be restored even if
        `do_munge` throws
        | finally(ex::unstoppable(restore_invariants(values...)));
    } );
}

auto sndr = ...;
spawn( sndr | safely_munge_data(), scope_token ); // See
`counting_scope` from P3149R6

```

§ 4 Proposed Wording

[Editor's note: The wording in this section is based on the current working draft.]

[Editor's note: Change [exec.syn] as follows:]

```

inline constexpr unspecified write_env{};
inline constexpr unspecified unstoppable{};
inline constexpr start_on_t start_on{};
inline constexpr continue_on_t continue_on{};
inline constexpr on_t on{};
inline constexpr schedule_from_t schedule_from{};

```

[Editor's note: Replace all instances of “*write-env*” with “*write_env*”. After [exec.adapt.objects], add a new subsection “*execution::write_env* [exec.write.env]” and move the specification of the exposition-only *write-env* from [exec.snd.expos]/p40-43 into it with the following modifications:]

(34.9.11.?) **execution::write_env [exec.write.env]**

```

template<sender Sndr, queryable Env>
constexpr auto write-env(Sndr&& sndr, Env&& env); //
exposition-only

```

1. ~~*write-env*~~*write_env* is ~~an exposition-only~~ a sender adaptor that accepts a sender and a queryable object, and that returns a sender that, when connected with a receiver rcvr, connects the adapted sender with a receiver whose execution environment is the result of joining the queryable ~~argument-env~~object to the result of `get_env(rcvr)`.
2. Let ~~*write-env-t*~~ be an exposition-only empty class type.
3. Returns: ~~*make-sender(write-env-t(), std::forward<Env>(env), std::forward<Sndr>(sndr))*~~.
2. *write_env* is a customization point object. For some subexpressions *sndr* and *env*, if `decltype((sndr))` does not satisfy *sender* or if `decltype((env))` does not satisfy *queryable*, the expression

`write_env(sndr, env)` is ill-formed. Otherwise, it is expression-equivalent to `make_sender(write_env, env, sndr)`.

3. **Remarks:** Let `write-env-t` denote the type `decltype(auto(write_env))`. The exposition-only class template `impls-for` ([exec.snd.expos]) is specialized for `write-env-t` as follows:

```
template<>
struct impls-for<write-env-t> : default-impls {
    static constexpr auto get_env =
        [](auto, const auto& env, const auto& rcvr) noexcept {
            return see below;
        };
};
```

Invocation of `impls-for<write-env-t>::get_env` returns an object `e` such that

- (43.1) — `decltype(e)` models *queryable* and
- (43.2) — given a query object `q`, the expression `e.query(q)` is expression-equivalent to `state.query(q)` if that expression is valid; otherwise, `e.query(q)` is expression-equivalent to `FWD-ENV(get_env(rcvr)).query(q)`.

[Editor's note: After [exec.write.env], add a new subsection “`execution::unstoppable` [exec.unstoppable]” as follows:]

(34.9.11.?) **`execution::unstoppable` [exec.unstoppable]**

1. `unstoppable` is a sender adaptor that connects its inner sender with a receiver that has the execution environment of the outer receiver but with a `never_stop_token` as the value of the `get_stop_token` query.
2. For a subexpression `sndr`, `unstoppable(sndr)` is expression-equivalent to `write_env(sndr, prop(get_stop_token, never_stop_token{}))`.

§ 5 References

[P3175R3] Eric Niebler. 2024-06-25. Reconsidering the ``std::execution::on`` algorithm. <https://wg21.link/p3175r3>